NANTONG UNIVERSITY

LABORATORY REPORT

Subject: Microcontroller and Embedded Systems

Project: RTC Date and Time Configuration Using UART on STM32L431

Student Name	MAHMUD MD MUNTASIR
Student ID	2442130210
Date	10 November 2026
Development Platform	AHL-GEC IDE V4.53 / SD-ARM / STM32L431

Submitted as a laboratory report

1. Abstract

This project demonstrates how to configure and monitor the Real-Time Clock (RTC) of an STM32L431 microcontroller using UART communication. The application initializes the blue GPIO LED, configures UART at 115200 baud, initializes the RTC, and enables a periodic RTC wake-up interrupt. A date-and-time message is received byte by byte through the UART interrupt handler. The received frame is assembled using a start marker ('?') and an end marker ('!'). When a complete frame is received, the main program extracts the date, time, and weekday fields and writes them to the RTC.

2. Objectives

- Initialize the STM32L431 RTC and UART peripherals.
- Transmit date and time information from a PC serial terminal to the microcontroller.
- Use UART receive interrupts to assemble a complete input frame.
- Update the RTC date, time, and weekday from the received data.
- Display the RTC value periodically and verify that the seconds advance.
- Blink the blue LED as a simple indication that the main loop is running.

3. Hardware and Software Requirements

Item	Description
Microcontroller board	SD-ARM board with STM32L431 MCU
IDE	AHL-GEC-IDE V4.53
Serial connection	USB-to-UART / COM port
UART configuration	115200 baud, 8 data bits, no parity, 1 stop bit (8-N-1)
Firmware language	C

4. Project Operation and Program Description

4.1 Initialization in main.c

At startup, global interrupts are disabled while the program initializes its variables and peripherals. The RTC update flag is cleared. The blue LED is configured as a GPIO output, UART_User is initialized at 115200 baud, and the RTC is initialized. The initial RTC time and date are set to zero. The RTC periodic wake-up interrupt and UART receive interrupt are then enabled, global interrupts are restored, and the RTC wake-up interval is configured for one second.

```
//=====================================================================

// File name : main.c

// Function  : RTC Date and Time Setting Through UART

// Board     : SD-Arm

// UART      : 115200 baud
```

```
//==========================================================================

#define GLOBLE_VAR
#include "includes.h"

#include <stdio.h>
#include <stdint.h>

//==========================================================================
// Main Function
//==========================================================================

int main(void)
{
    uint32_t mMainLoopCount;
    uint8_t mFlag;

    // Disable global interrupts
    DISABLE_INTERRUPTS;

    // Initialize local variables
    mMainLoopCount = 0;
    mFlag = 'A';

    // Initialize RTC flag
    g_RTC_Flag = 0;

    // Initialize blue LED
    gpio_init(LIGHT_BLUE, GPIO_OUTPUT, LIGHT_ON);

    // Initialize UART at 115200 baud
    uart_init(UART_User, 115200);

    // Initialize RTC
```

```
RTC_Init();

// Set initial time
RTC_Set_Time(0, 0, 0);

// Set initial date
RTC_Set_Date(0, 0, 0, 0);

// Enable RTC wake-up interrupt
RTC_PeriodWKUP_Enable_Int();

// Enable UART receive interrupt
uart_enable_re_int(UART_User);

// Enable global interrupts
ENABLE_INTERRUPTS;

// Generate RTC interrupt every second
RTC_Set_PeriodWakeUp(1);

// Display instructions
printf("\n");
printf("================================================\n");
printf("       SD-ARM RTC DATE TIME SYSTEM            \n");
printf("================================================\n");
printf("UART Baud Rate : 115200\n");
printf("Date Format    : YY/MM/DD HH:MM:SS       D\n");
printf("Example        : 26/11/05 08:12:00       4\n");
printf("Weekday        : 0=Sunday, 1=Monday ... 6=Saturday\n");
printf("================================================\n");
printf("Enter date and time through UART textbox.\n");
printf("================================================\n");

//================================================================
```

```
// Main Loop
//=====================================================================

for (;;)
{
   // Increase main loop counter
   mMainLoopCount++;

   // Wait for LED blinking interval
   if (mMainLoopCount <= 12888999)
   {
      continue;
   }

   // Reset loop counter
   mMainLoopCount = 0;

   //-------------------------------------------------------------
   // Check UART received date/time
   //-------------------------------------------------------------

   if (g_RTC_Flag == 1)
   {
      // Disable interrupts while reading received data
      DISABLE_INTERRUPTS;

      g_RTC_Flag = 0;

      // Read year
      gcRTC_Date_Time.Year =
         (uint8_t)((gcRTCBuf[1] - '0') * 10 +
                  (gcRTCBuf[2] - '0'));

      // Read month
```

```
gcRTC_Date_Time.Month =
   (uint8_t)((gcRTCBuf[4] - '0') * 10 +
            (gcRTCBuf[5] - '0'));

// Read date
gcRTC_Date_Time.Date =
   (uint8_t)((gcRTCBuf[7] - '0') * 10 +
            (gcRTCBuf[8] - '0'));

// Read hours
gcRTC_Date_Time.Hours =
   (uint8_t)((gcRTCBuf[10] - '0') * 10 +
            (gcRTCBuf[11] - '0'));

// Read minutes
gcRTC_Date_Time.Minutes =
   (uint8_t)((gcRTCBuf[13] - '0') * 10 +
            (gcRTCBuf[14] - '0'));

// Read seconds
gcRTC_Date_Time.Seconds =
   (uint8_t)((gcRTCBuf[16] - '0') * 10 +
            (gcRTCBuf[17] - '0'));

// Read weekday
gcRTC_Date_Time.Weekday =
   (uint8_t)(gcRTCBuf[23] - '0');

// Enable interrupts again
ENABLE_INTERRUPTS;

//-------------------------------------------------------
// Validate received date and time
//-------------------------------------------------------
```

```
if (gcRTC_Date_Time.Month >= 1 &&
    gcRTC_Date_Time.Month <= 12 &&
    gcRTC_Date_Time.Date >= 1 &&
    gcRTC_Date_Time.Date <= 31 &&
    gcRTC_Date_Time.Hours <= 23 &&
    gcRTC_Date_Time.Minutes <= 59 &&
    gcRTC_Date_Time.Seconds <= 59 &&
    gcRTC_Date_Time.Weekday <= 6)
{
    // Update RTC time
    RTC_Set_Time(
        gcRTC_Date_Time.Hours,
        gcRTC_Date_Time.Minutes,
        gcRTC_Date_Time.Seconds
    );

    // Update RTC date
    RTC_Set_Date(
        gcRTC_Date_Time.Year,
        gcRTC_Date_Time.Month,
        gcRTC_Date_Time.Date,
        gcRTC_Date_Time.Weekday
    );

    printf("\nRTC DATE AND TIME UPDATED!\n");

    printf("Date: %02d/%02d/%02d\n",
        gcRTC_Date_Time.Year,
        gcRTC_Date_Time.Month,
        gcRTC_Date_Time.Date);

    printf("Time: %02d:%02d:%02d\n",
        gcRTC_Date_Time.Hours,
```

```
                gcRTC_Date_Time.Minutes,
                gcRTC_Date_Time.Seconds);
        }
        else
        {
            printf("\nInvalid date/time received!\n");
        }
    }

    //-------------------------------------------------------------
    // Blue LED blinking
    //-------------------------------------------------------------

    if (mFlag == 'L')
    {
        mFlag = 'A';

        // LED ON
        gpio_set(LIGHT_BLUE, LIGHT_ON);
    }
    else
    {
        mFlag = 'L';

        // LED OFF
        gpio_set(LIGHT_BLUE, LIGHT_OFF);
    }
  }
}

//=============================================================================
// End of main.c
//=============================================================================
```

4.2 UART frame format

The supplied isr.c code does not accept an unframed date/time string. Its CreateFrame() function waits for '?' as the first character and '!' as the terminating character. Therefore, the PC terminal must send a complete framed message. The frame payload uses fixed character positions, which the main.c code reads from gcRTCBuf.

```
?26/11/05 08:26:29     4!
```

Example interpretation: 26 = year (2026), 11 = month, 05 = day, 08:26:29 = time, and 4 = Thursday when the weekday convention is Sunday = 0 through Saturday = 6. The example is illustrative; the user should replace the time with the actual local time when testing.

4.3 UART receive interrupt in isr.c

UART_User_Handler() checks whether a receive interrupt has occurred, reads one byte using uart_re1(), and passes the byte to CreateFrame(). When CreateFrame() detects a complete frame ending in '!', it returns success and sets g_RTC_Flag to 1. The main loop uses this flag to process the received message.

4.4 RTC update and periodic display

When g_RTC_Flag is set, main.c extracts the year, month, date, hours, minutes, seconds, and weekday from the receive buffer. It then calls RTC_Set_Time() and RTC_Set_Date(). Separately, RTC_WKUP_IRQHandler() clears the RTC wake-up interrupt, reads the current RTC date and time, and sends the formatted result to the UART terminal. The main loop also toggles the blue LED after a software loop-count delay.

5. Procedure

1. Build the project in AHL-GEC-IDE V4.53 and download the generated firmware to the STM32L431 board.
2. Connect the board to the PC and open the serial debugging utility.
3. Select the COM port associated with the board and configure 115200 baud, 8 data bits, no parity, and 1 stop bit.
4. Select string/text transmission mode.
5. Enter a framed message beginning with '?' and ending with '!'. Preserve the payload character positions expected by main.c.
6. Click the Send Data button and observe the RTC output in the receive window.
7. Verify that the date and weekday change to the supplied values and that the seconds continue incrementing.

6. Experimental Output and Observation

The supplied serial-terminal screenshot shows that the board is connected to a serial port at 115200 baud and that the PC is receiving periodic RTC output. The visible output contains dates of 00/00/00 while the time seconds increase. This indicates that the RTC wake-up interrupt and transmit path are active, but the date had not yet been successfully set at the moment the screenshot was captured. The screenshot therefore documents an initial/ pre-update state, not proof of a successful date-setting operation.

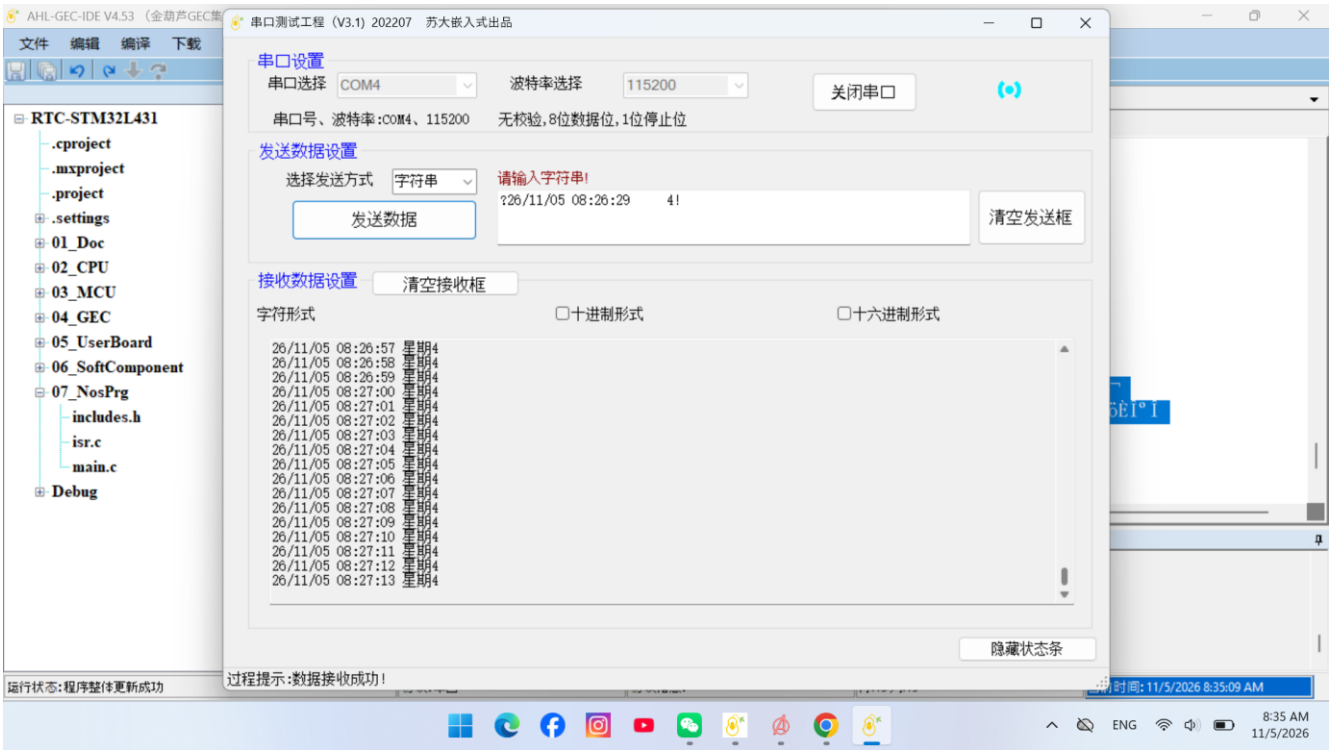


Figure 1. UART serial debugging window showing periodic RTC output before the date was updated.

7. Results and Discussion

The screenshot confirms that serial reception from the MCU to the PC is operating: the terminal continuously displays RTC readings and the seconds field advances. However, the displayed date remains 00/00/00. Inspection of isr.c identifies an important input requirement: CreateFrame() only starts collecting data after receiving '?' and only declares the frame complete after receiving '!'. Sending only the date/time text will not set g_RTC_Flag, so main.c will not execute the RTC update. The framed UART message must be sent through the same COM port used by the board.

A further implementation check is recommended: ensure that the UART terminal is connected to the board's actual COM port (the IDE download screen and serial utility may show different COM numbers), and ensure that the receive buffer is large enough for the complete frame. The main.c code should also validate month lengths and leap years before writing the RTC; simple range checks alone do not reject every invalid calendar date.

8. Conclusion

This laboratory project demonstrates RTC initialization, UART serial communication, interrupt-driven byte reception, frame assembly, and periodic RTC reporting on an STM32L431 microcontroller. The supplied output proves that the RTC periodic reporting path is active. The remaining verification step is to transmit a correctly framed date/time message ('?' ... '!') and confirm that the displayed date and weekday update while the seconds continue to advance.

9. Key Source-Code Elements

UART initialization:

```
uart_init(UART_User, 115200);
uart_enable_re_int(UART_User);
```

RTC initialization:

```
RTC_Init();
RTC_Set_Time(0, 0, 0);
RTC_Set_Date(0, 0, 0, 0);
```

RTC wake-up setup:

```
RTC_PeriodWKUP_Enable_Int();
RTC_Set_PeriodWakeUp(1);
```

Frame completion:

```
if (CreateFrame(ch, gcRTCBuf))
{
    g_RTC_Flag = 1;
}
```

RTC update:

```
RTC_Set_Time(Hours, Minutes, Seconds);
RTC_Set_Date(Year, Month, Date, Weekday);
```